

File Organisation I: Sequential and Direct Files

CEN207 Data Structures — Week 13

Asst. Prof. Dr. Uğur CORUH · Fall 2026–2027

Today's plan (3 hours)

Hour	Topic
1	Blocks, records/fields Anim 1 · blocking factor Anim 2 · sequential search Anim 3
2	Binary search of a sorted file Anim 4 · sequential update / merge Anim 5
3	Relative access Anim 6 · hashing to buckets Anim 7 · progressive overflow Anim 8 · tombstones Anim 9

Learning outcomes: LO.1 (explain fundamental data structures) · LO.2 (analyze complexity) · LO.6 (implement in C and Java) · LO.7 (choose the right structure)

This week's concepts — where

Concept	Where
Records/fields, blocking factor	Sections 2–3
Sequential search, binary search of a sorted file	Sections 4–5
Sequential update (merge master + transactions)	Section 6
Relative (direct) access	Section 7
Hashing to buckets, progressive overflow, tombstones	Sections 8–10

How the code examples work

- Every idea has a complete **C** and **Java** program
- Both perform **real file I/O**: `fopen` / `fseek` / `fread` / `fwrite` , `RandomAccessFile`
- Each program creates its own **temporary lab folder**, then deletes it
- Sources: `code/week-13/c/` and `code/week-13/java/`
- Each program's expected output is in the week notes

Recap — computed addresses (Week 1)

- `arr[i] = base_address + i * element_size`
- No **searching** needed to reach index `i` — just arithmetic
- Section 7's relative file is exactly this idea, moved to disk
- `block = rrn / bf`, `offset = rrn % bf` — computed, not searched

Recap — hashing and collisions (Week 6)

- $h(\text{key}) = \text{key} \bmod m$ picks a slot in RAM
- Collisions resolved by **chaining** or **open addressing**
- Sections 8–10 reuse both ideas, unchanged in logic
- The only difference: every "slot" is now a disk position

The one genuinely new idea

- Weeks 1–12: cost = **comparisons**, swaps, pointer hops
- This week: cost = **block reads and writes**
- A block read costs orders of magnitude more than any RAM op
- Comparing extra records *already in RAM* costs almost nothing

Map of the week — at a glance

Family	Sections
Foundations	Records/fields (2), blocking factor (3)
Sequential — process in order	Search (4), sorted search (5), update (6)
Direct — compute the location	Relative (7), hashing (8–10)

1. Why Files Are Different: Block-Oriented I/O

A question to start

A file usually cannot fit in RAM at all. If a program needs one 20-byte record, why does the disk not just hand over 20 bytes?

The idea: a block is the smallest unit of I/O

- Disks are organized into fixed-size **blocks**
- Every read/write transfers **one whole block**, never less
- Even a 20-byte request loads the entire surrounding block
- This is *why* file organisation exists as a topic at all

This week's drawing convention

- A **disk** row: named, numbered blocks, contents drawn inside
- A **RAM buffer** row: the one block currently loaded
- **Block reads** and **block writes**, counted on the right
- That count *is* this week's `0(...)` — watch it, not the keys

Two families, one week

Family	Locate a record
Sequential (sections 4–6)	Read blocks in order, or jump to a middle block
Direct (sections 7–10)	One calculation, then one block read

Sequential wins when you process **every** record anyway; direct wins for **one** lookup among millions.

Mini-quiz

A sequential search reads 100 blocks, comparing 1 key per block. A second file reads 100 blocks, comparing 1000 keys per block. Which is faster?

Answer

Neither — they cost about the same. The disk access dominates; comparing extra keys already sitting in a loaded block is nearly free.

2. Records and Fields

A question to start

A name field can hold "Al" or "Christopherson". Should every record reserve the same number of bytes for it regardless?

Three layouts for the same field

Layout	How	Cost
Fixed-length	Pad or truncate to <code>NAME_FIXED</code>	Wasted space or lost data
Delimited	Write value, then a delimiter byte	Scan to find the delimiter
Length-prefixed	Write length byte, then that many bytes	$O(1)$ read, max length 255

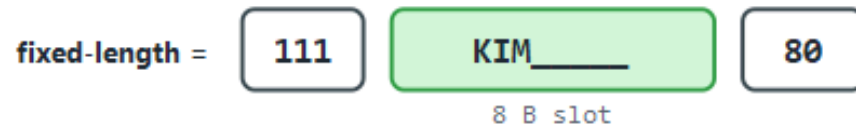
Why fixed-length matters most

- Every record is **exactly the same size**
- That is what makes computing an address from a position possible
- Section 3 (blocking) and section 7 (direct access) both need this
- Delimited/length-prefixed records cannot be located this way

Intuition — a form with a "name" box

- A paper form's name box is a **fixed** size on the page
- A spreadsheet cell **grows** to fit whatever you type — delimited
- A shipping label prints a length, then that many characters
- All three exist in real systems, for exactly these trade-offs

Records and fields, step by step



total = 176 B (wasted 55 B)



total = 132 B

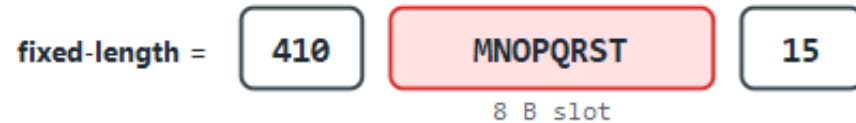


total = 132 B

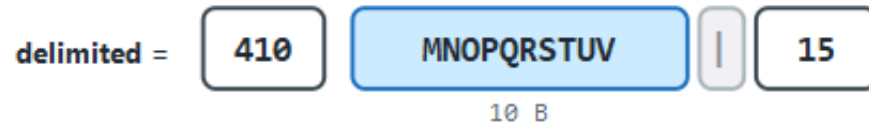
0 record(s) truncated; 55 bytes wasted in the fixed format in total.

Done: 11 records. Fixed = 176 B (55 B wasted, 0 truncation(s)), delimited = 132 B, length-prefixed = 132 B. Fixed-length is simpler to read back but wastes space or loses data; the other two carry the same information exactly.

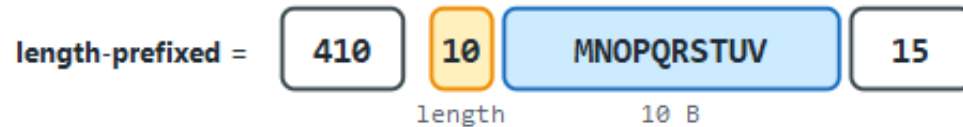
Edge case — every name longer than 8 characters



total = 160 B (wasted 0 B)



total = 190 B

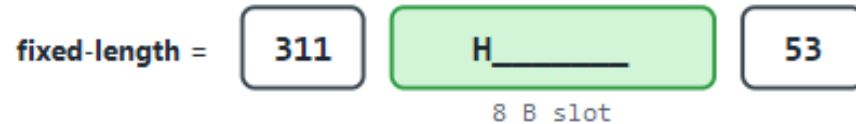


total = 190 B

10 record(s) truncated; 0 bytes wasted in the fixed format in total.

Done: 10 records. Fixed = 160 B (0 B wasted, 10 truncation(s)), delimited = 190 B, length-prefixed = 190 B. Fixed-length is simpler to read back but wastes space or loses data; the other two carry the same information exactly.

Edge case — an empty name and a very long name



total = 176 B (wasted 50 B)



total = 147 B



total = 147 B

4 record(s) truncated; 50 bytes wasted in the fixed format in total.

Done: 11 records. Fixed = 176 B (50 B wasted, 4 truncation(s)), delimited = 147 B, length-prefixed = 147 B. Fixed-length is simpler to read back but wastes space or loses data; the other two carry the same information exactly.

Code — fixed-length: pad or truncate

```
#define NAME_FIXED 8

int n = (int) strlen(name);
if (n >= NAME_FIXED) {
    /* too long: truncate, data is LOST */
    memcpy(r.name, name, NAME_FIXED);
} else {
    memcpy(r.name, name, n);
    /* pad with filler bytes */
    memset(r.name + n, '_', NAME_FIXED - n);
}
```

Code — length-prefixed: $O(1)$ to read back

```
unsigned char len =  
    (unsigned char) strlen(name);  
fwrite(&len, 1, 1, fp);    /* 1-byte prefix */  
fwrite(name, 1, len, fp);  
  
/* reading back: read len, then read  
   exactly len more bytes – no scanning */
```

Complexity

- Writing/reading one field: $O(\text{field length})$, all three layouts
- The difference is **not** asymptotic
- It is wasted space (fixed) vs. parsing cost (delimited) vs. neither (capped, length-prefixed)

Common mistakes

- Forgetting a full/truncated fixed field has **no room for** `'\0'`
- Choosing a delimiter that can legally appear inside the data
- **Silently** truncating instead of reporting the data loss

Mini-quiz

Why is a length-prefixed field's read cost $O(1)$ plus its own length, while a delimited field's read cost is a full scan?

Answer

The length-prefixed reader is told the size in advance (one byte, then that many bytes).

A delimited reader must scan byte by byte, comparing against the delimiter, to find the end.

3. Blocking Factor

A question to start

A block is 100 bytes; one record is 20 bytes.
Does the disk waste 80 bytes on every read, or
can several records share one block?

The idea: pack `bf` records per block

- `bf = floor(blockSize / recSize)`
- Records fill a small **RAM buffer**, one at a time
- The moment the buffer reaches `bf` records: **one** `fwrite`
- Disk access happens **per block**, not per record

Two kinds of waste

Kind	Where	Cause
Internal fragmentation	Every full block	<code>bf * recSize</code> rarely equals <code>blockSize</code> exactly
Last-block waste	Only the final block	Record count is not an exact multiple of <code>bf</code>

Intuition — a moving truck, not a courier bike

- A courier bike carries **one** package per trip: slow, one at a time
- A moving truck carries `bf` boxes per trip: fewer trips, same boxes
- A truck that leaves half-empty still burns a full trip's fuel
- That "half-empty trip" is exactly this section's last-block waste

Blocking factor, step by step



blocks written = 3
internal waste = 0 B
last-block waste = 40 B

Done: 13 keys → 3 block(s) written. Without a blocking factor (bf = 5, one disk access per record) this would take 13 accesses; now only 3. Internal waste totals 0 B, last-block waste 40 B.

Edge case — record bigger than the block

ERROR: the record (60 B) is bigger than the block (50 B) — not even one record fits!

disk =

blocks written = 0

internal waste = 0 B

last-block waste = 0 B

RAM buffer =

bf = 0 — this is a design error: the block size must be increased or the record shrunk. No write happens.

Edge case — 12 keys divide bf=3 exactly



Done: 12 keys → 4 block(s) written. Without a blocking factor (bf = 3, one disk access per record) this would take 12 accesses; now only 4. Internal waste totals 40 B, last-block waste 0 B.

Code — capacity, fill, and flush

```
#define BLOCK_SIZE 100

int block_capacity(int rec_size) {
    return BLOCK_SIZE / rec_size;
}

/* block_put: append a key; flush when full */
/* block_flush: write a PARTIAL last block */
/*   anyway – a whole block, however few keys */
```

Complexity

- Blocking itself changes no algorithm's **asymptotic** cost
- It changes the **constant**: block reads/writes, not record count
- Larger `bf` → fewer, bigger disk accesses → faster in practice
- Up to the point a block becomes too large to buffer in RAM

Common mistakes

- Forgetting the **last block** still costs a full block write
- Picking `blockSize` smaller than one record (`bf = 0`)
- Confusing internal fragmentation with last-block waste — they are independent

Mini-quiz

`recSize = 24`, `blockSize = 100`. What is `bf`, and how many bytes of internal fragmentation does **one full block** waste?

Answer

bf = 4 . **4 * 24 = 96** bytes used,
4 bytes wasted per full block — small here,
but multiplied by every block in a huge file.

4. Sequential Search of a File

A question to start

A file's records are not sorted. To find one key, is there any option other than reading every block until it turns up, or the file ends?

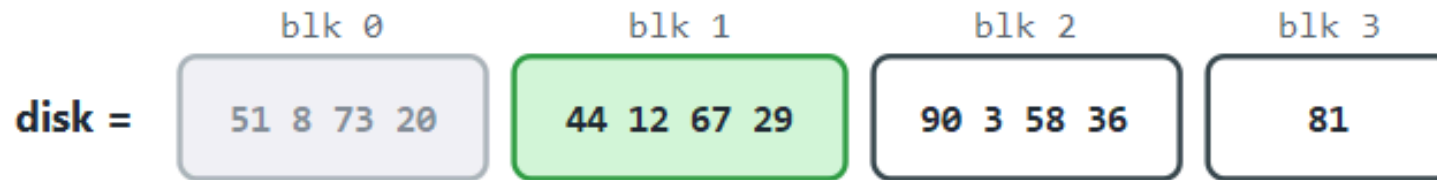
The idea: read blocks in order, compare inside

- Read block 0 into the RAM buffer
- Compare **every** record inside it against the target
- Not found? Read block 1 — and so on
- Cost that matters: **block reads**, not comparisons

Intuition — reading a shelf of unsorted boxes

- Looking for one file in an **unsorted** shelf of boxes
- You must open **every** box, one by one, until you find it
- Or open every box and find nothing — you still opened them all
- A "box" here is one block; opening it is one disk read

Sequential search, step by step



target = 67

block reads = 2

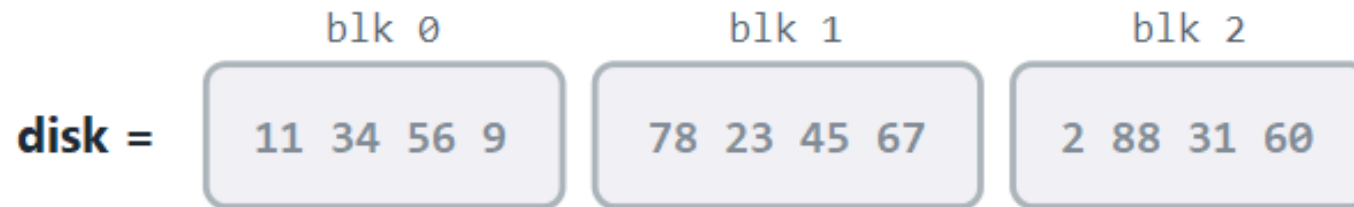
comparisons = 7

found, position 6



Done: 67 found at position 6 — 2 block read(s) (out of 4 blocks), 7 comparisons. On average the expected read count is $\approx \text{numBlocks}/2$; the worst case reads every block.

Edge case — target absent, whole file scanned



target = 999

block reads = 3

comparisons = 12

not found



Done: 999 not found — the whole file was scanned: 3 block reads, 12 comparisons. Not-found is always the worst case.

Edge case — target is the first record (best case)



target = 70

block reads = 1

comparisons = 1

found, position 0



Done: 70 found at position 0 — 1 block read(s) (out of 3 blocks), 1 comparisons. On average the expected read count is $\approx \text{numBlocks}/2$; the worst case reads every block.

Code — scan one block, then the next

```
for (int b = 0; b < nblocks; b++) {
    load_block(f, b, buf, &cnt);
    (*block_reads)++;
    for (int i = 0; i < cnt; i++) {
        (*comparisons)++;
        if (buf[i] == target)
            return b * bf + i;
    }
}
return -1; /* exhausted every block */
```

Complexity

- Worst case: $\text{ceil}(n / bf)$ block reads — every block
- Average case: about **half** that
- The exact file analogue of Week 1's $O(n)$ linear search
- Unit counted is **blocks**, not records — $O(n / bf)$

Common mistakes

- Counting comparisons instead of **block reads** as the real cost
- Assuming a key is absent before reading **every** remaining block
- Returning the **last** match of a duplicate instead of the first

Mini-quiz

Why is sequential search's worst case exactly the same whether the target is absent, or is the very last record in the file?

Answer

Both require reading every block. For the last record, the match is not found until the final block; for an absent key, nothing rules out any block early — every one must be checked.

5. Binary Search of a Sorted File

A question to start

If section 4's file is kept **sorted**, can it be searched the way Week 1's binary search searches a sorted array — but block by block?

The idea: binary search over BLOCKS

- Block **b**'s records are a sorted, non-overlapping **range**
- Compare target to a block's **first** and **last** key
- Too small? Discard the right half of the block range
- Too big? Discard the left half — otherwise, it is **in** this block

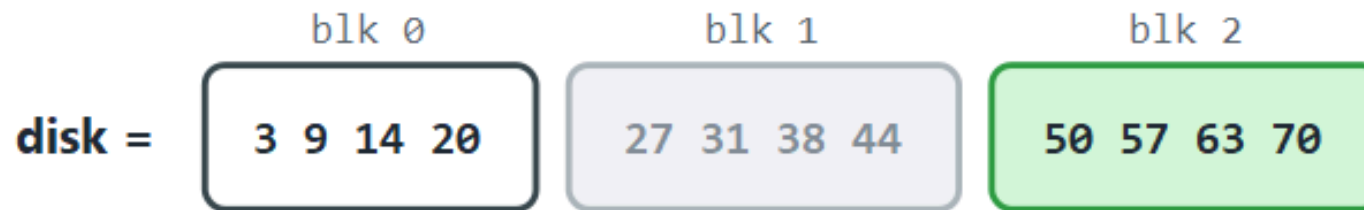
What "gap" means

- "In range" does **not** mean "present"
- A target between a block's first and last key might still be **absent**
- The linear scan inside that one block confirms or denies it
- This never happens in section 4 — an unsorted file has no ranges

Intuition — a labeled shelf of sorted boxes

- Now the shelf's boxes are **labeled** with their contents' range
- Check one box's label: your file is before it, inside it, or after
- Skip straight past every box that cannot possibly hold it
- Only open the **one** box whose label range matches

Binary search of a sorted file, step by step



target = 63

block reads = 2

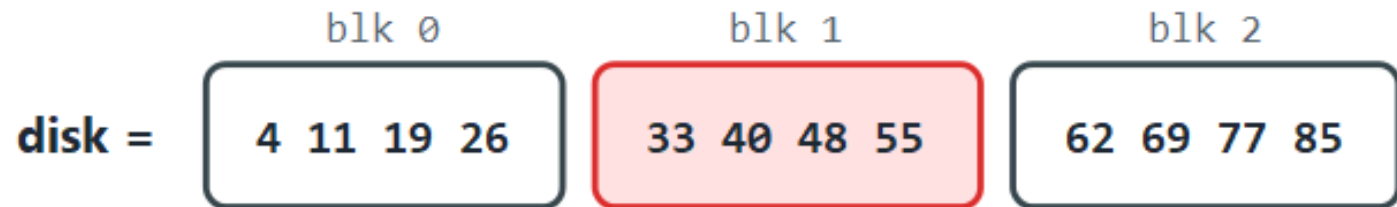
lo..hi = 2..2

63 == 63



Done: 63 found at position 10 — only 2 block read(s) (out of 3; sequential search would need up to 3 in the worst case). $O(\log \text{ numBlocks}) \approx 2$.

Edge case — a gap inside a block's range



target = 45

block reads = 1

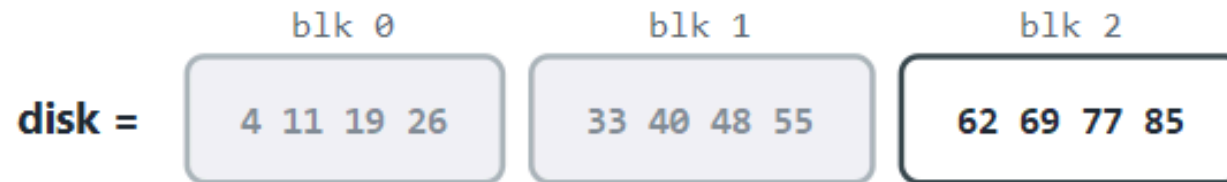
lo..hi = 0..2

55 != 45



Done: 45 not found — decided with 1 block read(s) (sequential search would need 3).

Edge case — target below the file's minimum key



target = 1

block reads = 2

lo..hi = 0..-1

not found (out of range)



Done: 1 not found — decided with 2 block read(s) (sequential search would need 3).

Code — narrow to one block, then scan

```
while (lo <= hi) {  
    int mid = (lo + hi) / 2;  
    block_reads++;  
    if (target < first_key(mid)) hi = mid-1;  
    else if (target > last_key(mid)) lo = mid+1;  
    else {  
        /* target's range: short scan inside */  
        return scan_block(mid, target);  
    }  
}
```

Complexity

- $O(\log_2 \text{ numBlocks})$ block reads, plus a scan of $\leq \text{bf}$ records
- Compare to section 4's $O(\text{numBlocks})$
- 24 keys, 6 blocks: 3 reads here vs. 6 for sequential
- Gap widens dramatically as the file grows

Common mistakes

- Forgetting "in range" does not mean "present" (the gap case)
- Comparing against the block's **middle** key instead of first/last
- Applying this to an **unsorted** file — silently wrong, not just slow

Mini-quiz

Array binary search (Week 1) needs $O(\log n)$ comparisons. File binary search needs $O(\log \text{ numBlocks})$ reads. Is one better?

Answer

Neither — same idea, different unit. Array search halves *elements*; file search halves *blocks*, and `numBlocks` is already `bf` times smaller than `n` to start with.

6. Sequential Update: Merging Master and Transactions

A question to start

A bank's account file changes daily. Rewriting the **entire** file for every change is absurd at millions of records. What is the alternative?

A short history

- The **sorted master + transaction file** merge is a classical batch-processing pattern, older than random-access disks
- Tharp's *File Organization and Processing* treats it as the canonical sequential-file update algorithm
- Still used today for huge, periodic batch jobs (billing runs)

The idea: merge two SORTED files, one pass

- **Transaction file**: sorted, same key, each with an **op**
- **'A'** add, **'C'** change, **'D'** delete
- Two "read heads", one per file — exactly Week 10's merge step
- Compare **master.key** to **txn.key** at every step

The three cases

Compare	Meaning	Action
<code>master.key < txn.key</code>	No transaction for this record	Copy unchanged
<code>master.key > txn.key</code>	Key not yet in master	'A' : insert; else error
<code>master.key == txn.key</code>	Transaction applies here	See next slide

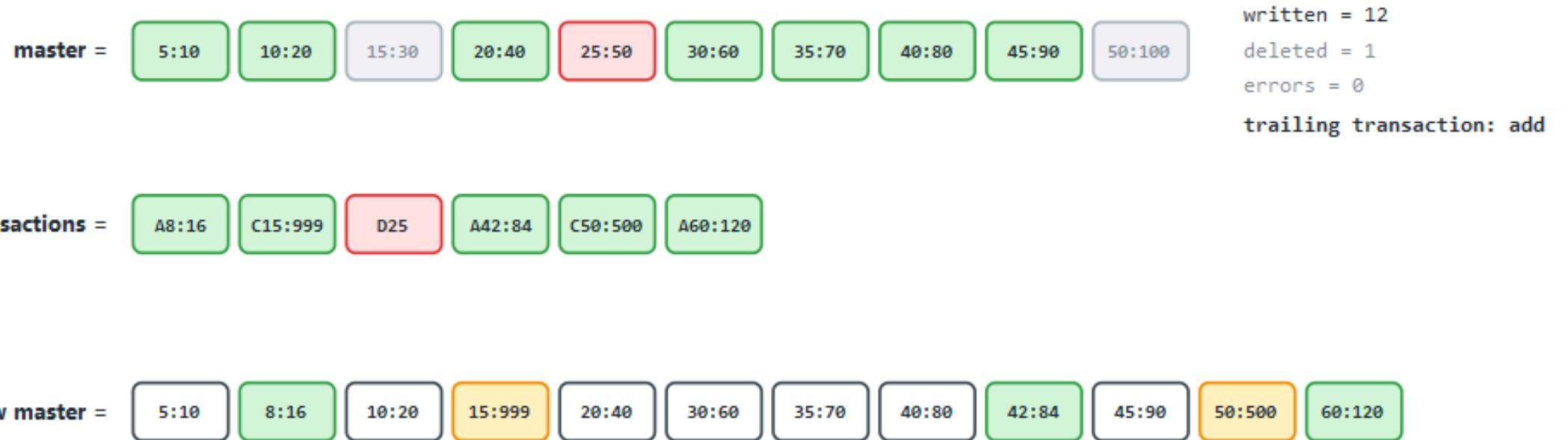
The equal-key case, in detail

- 'A' on an existing key: **duplicate-add error**
- The original record is **still copied through** unchanged
- 'C': value replaced, updated record written
- 'D': record simply **not written** — that is the delete

Intuition — two people reading two sorted lists

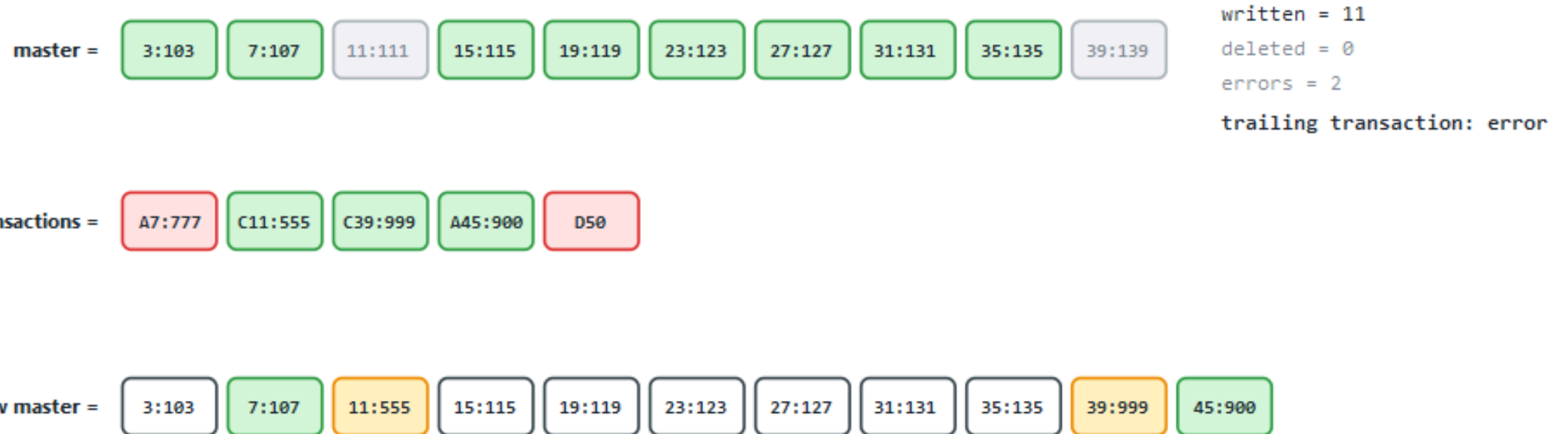
- One person reads the master list aloud, in order
- A second person reads the transaction list aloud, in order
- At every step, they compare their current word out loud
- Whoever is "behind" alphabetically reads their next word

Sequential update, step by step



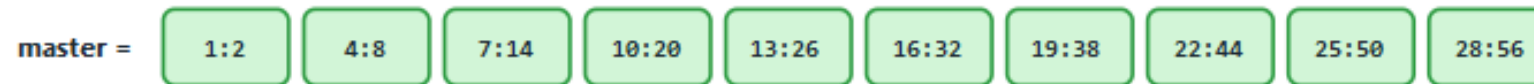
Done: 12 record(s) written to the new master, 1 deleted, 0 error(s). One pass, scanning both sorted files in parallel — $O(nm + nt)$.

Edge case — duplicate add + missing key



Done: 11 record(s) written to the new master, 0 deleted, 2 error(s). One pass, scanning both sorted files in parallel — $O(nm + nt)$.

Edge case — master runs out, trailing transactions

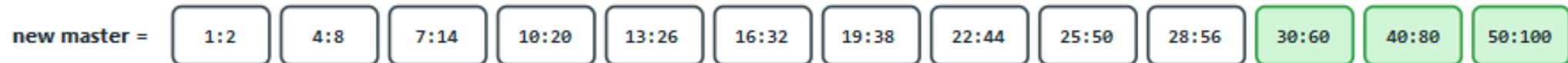
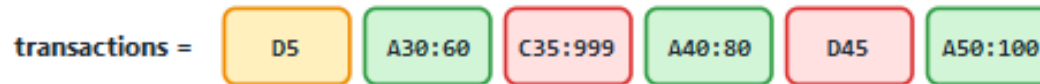


written = 13

deleted = 0

errors = 3

trailing transaction: add



Done: 13 record(s) written to the new master, 0 deleted, 3 error(s). One pass, scanning both sorted files in parallel — $O(nm + nt)$.

Code — the merge loop's core comparison

```
while (i < nm && j < nt) {  
    if (master[i].key < txn[j].key) {  
        write(master[i]); i++;  
    } else if (master[i].key > txn[j].key) {  
        if (txn[j].op == 'A') write(new);  
        else errors++;      /* C/D: missing */  
        j++;  
    } else { /* equal key: next slide */ }  
}
```

Code — the equal-key branch, and leftovers

```
else {
    if (txn[j].op == 'A') {
        write(master[i]); /* KEEP it */
        errors++;
    } else if (txn[j].op == 'C') write(new);
    else deleted++;      /* 'D': skip write */
    i++; j++;
}
/* + two leftover loops when one file ends */
```

Complexity

- $O(nm + nt)$ — one pass, every record read exactly once
- Compare to searching the master once per transaction: $O(nt * nm)$
- Section 4 or 5's search, applied nt times, would cost exactly that
- The merge is dramatically cheaper at any real scale

Common mistakes

- **Silently dropping** the master record on a duplicate-add error
- Forgetting the **two leftover loops** after the main merge ends
- Assuming both files are sorted, without checking or guaranteeing it

Mini-quiz

Why is a delete, in this algorithm, done by **not writing** a record, rather than writing a special "deleted" record?

Answer

The new file is built from scratch, record by record. A deleted record should not exist in it at all — skipping the write *is* the deletion, with no extra bookkeeping.

7. Relative (Direct) File Access

A question to start

A program already knows it wants "record number 7." Sections 4 and 5 both *search*. Does finding record 7 need any searching?

The idea: compute, do not search

- Every record gets a **relative record number (RRN)**: `0, 1, 2, ...`
- `block = rrn / bf`, `offset = rrn % bf`
- One division, one modulo — **no comparisons at all**
- One `fseek`, one block read — genuine **$O(1)$**

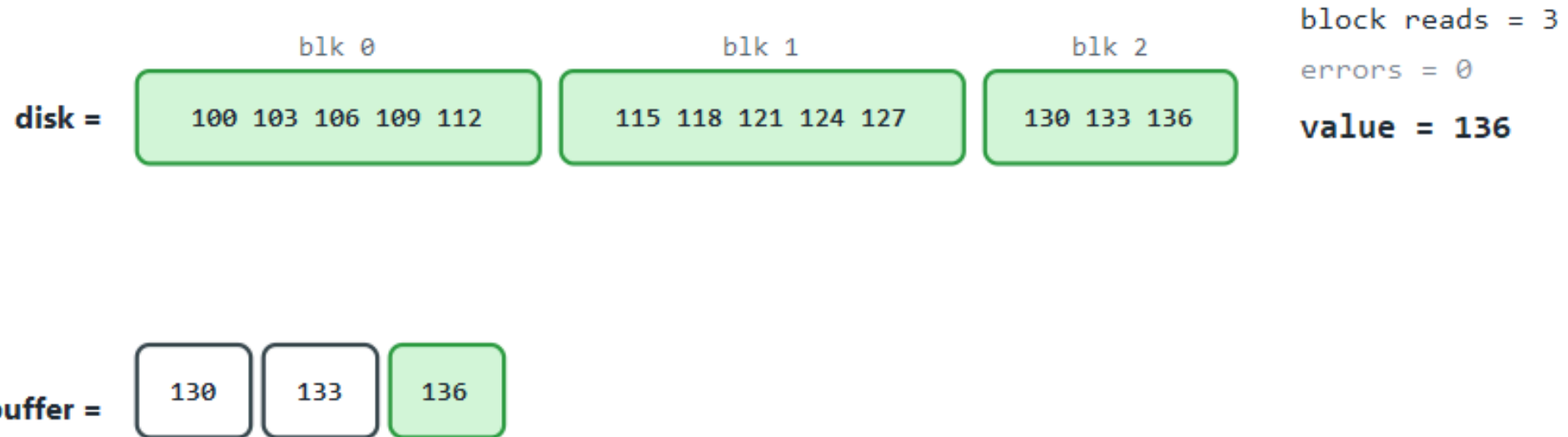
What must still be validated

- Negative `rrn`, or `rrn >= total`: reject **before** any read
- A valid-looking `rrn` can still land in a **partial last block**
- Must check against the **total record count**, not block bounds alone

Intuition — a numbered parking garage

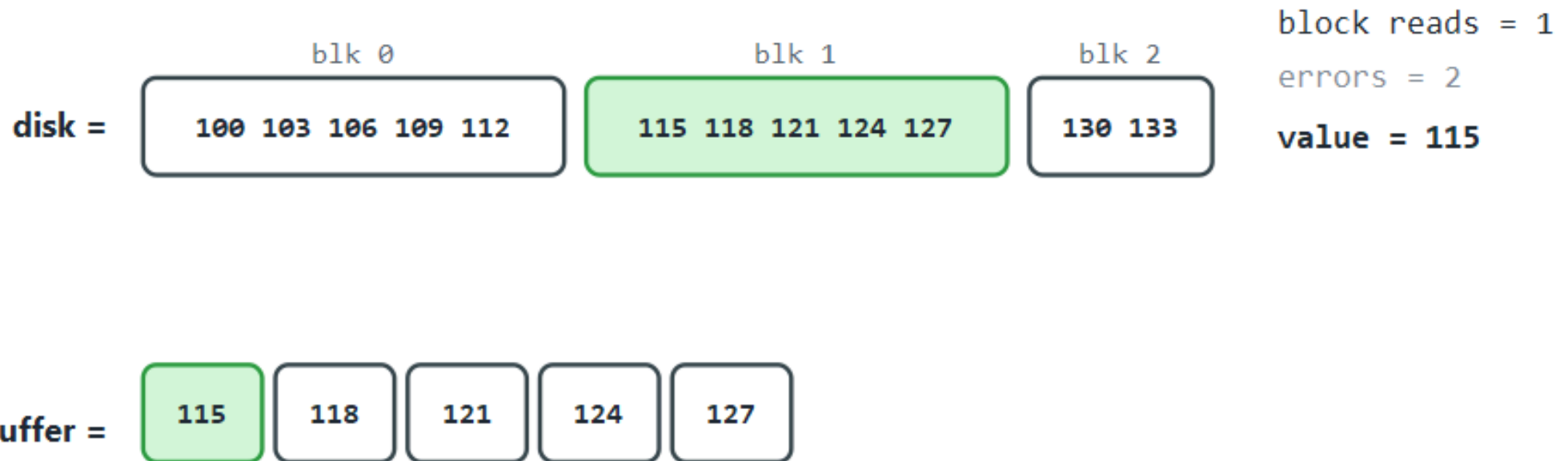
- Every parking space has a **number**, painted on the floor
- "Space 37" needs no searching — go straight to floor 37/bf
- A parking attendant never walks every floor looking for space 37
- Exactly section 7's $\text{block} = \text{rrn} / \text{bf}$, $\text{offset} = \text{rrn} \% \text{bf}$

Relative file access, step by step



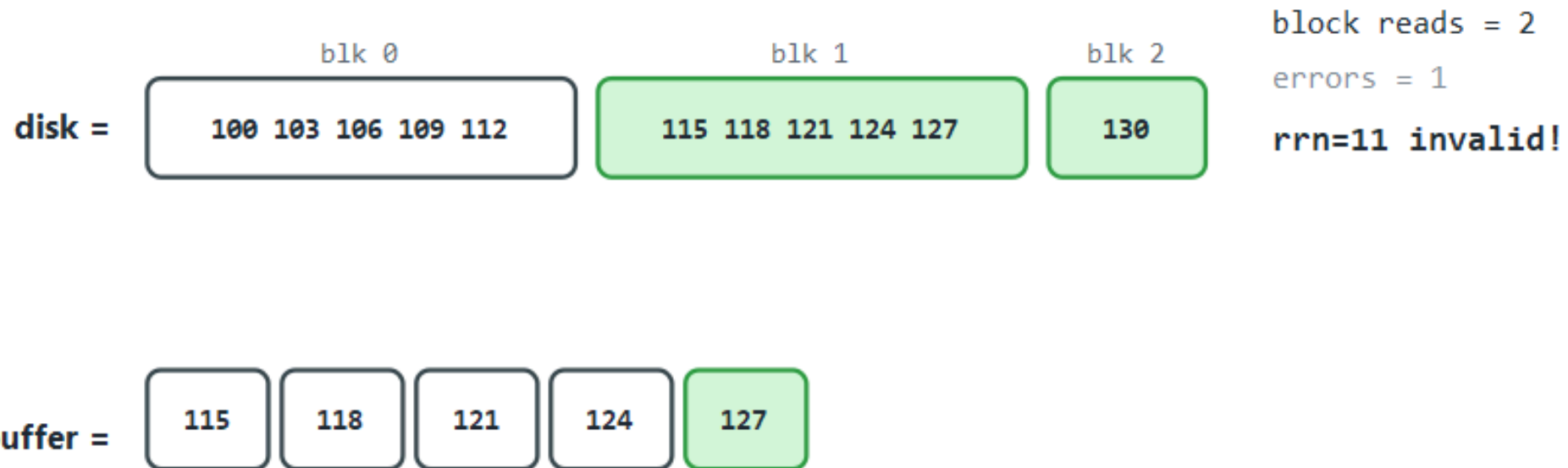
Done: 3 request(s), 3 block read(s) (exactly 1 per valid request), 0 error(s). Unlike sequential or binary search, relative access reads a fixed number of blocks regardless of file size.

Edge case — negative and out-of-range requests



Done: 3 request(s), 1 block read(s) (exactly 1 per valid request), 2 error(s). Unlike sequential or binary search, relative access reads a fixed number of blocks regardless of file size.

Edge case — the single record in a partial last block



Done: 3 request(s), 2 block read(s) (exactly 1 per valid request), 1 error(s). Unlike sequential or binary search, relative access reads a fixed number of blocks regardless of file size.

Code — the entire algorithm, in two lines

```
int direct_read(FILE *f, int bf,
                int total, int rrn) {
    if (rrn < 0 || rrn >= total)
        return -1;          /* reject */
    int block = rrn / bf;
    int offset = rrn % bf;
    /* one fseek, one fread, done */
}
```

Complexity

- $O(1)$ — exactly one block read, for every valid request
- Sections 4 and 5's costs both still **grow** as the file grows
- Direct access does **not** grow at all, ever
- The upper bound this whole chapter of techniques aims toward

Common mistakes

- Checking `rrn >= total` but forgetting `rrn < 0`
- Not accounting for a **partial last block**
- Assuming "direct" means "no validation needed" — it still does

Mini-quiz

Why does relative access's $O(1)$ cost not depend on `bf` at all, while sections 4 and 5's search costs both do?

Answer

Direct access always does exactly one seek and one read, regardless of `bf` — only *which* block changes. Searching must visit blocks one at a time until it finds the answer.

8. Hashing to Buckets

A question to start

Real keys are rarely a convenient `0, 1, 2, ...`.

Week 6 solved this in RAM with a hash function. Can the same idea work for a file?

The idea: hash to a BUCKET, chain overflow

- $h(\text{key}) = \text{key} \bmod m$ picks one of m buckets
- A bucket is a **block**, holding up to bf keys, not one
- Two keys, same bucket: fine, **until** the bucket is full
- Only then: allocate an **overflow block**, chain it on

Intuition — a row of mailboxes, each holding several letters

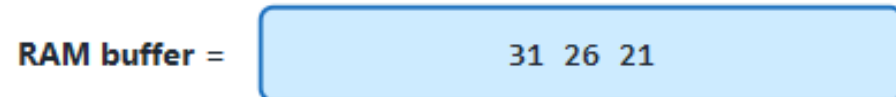
- Each mailbox (bucket) already holds up to `bf` letters (keys)
- Two letters for the same box: fine, until the box is full
- A full mailbox gets an **extra tray** clipped on beside it
- The extra tray is exactly one overflow block, chained on

Hashing to buckets, step by step



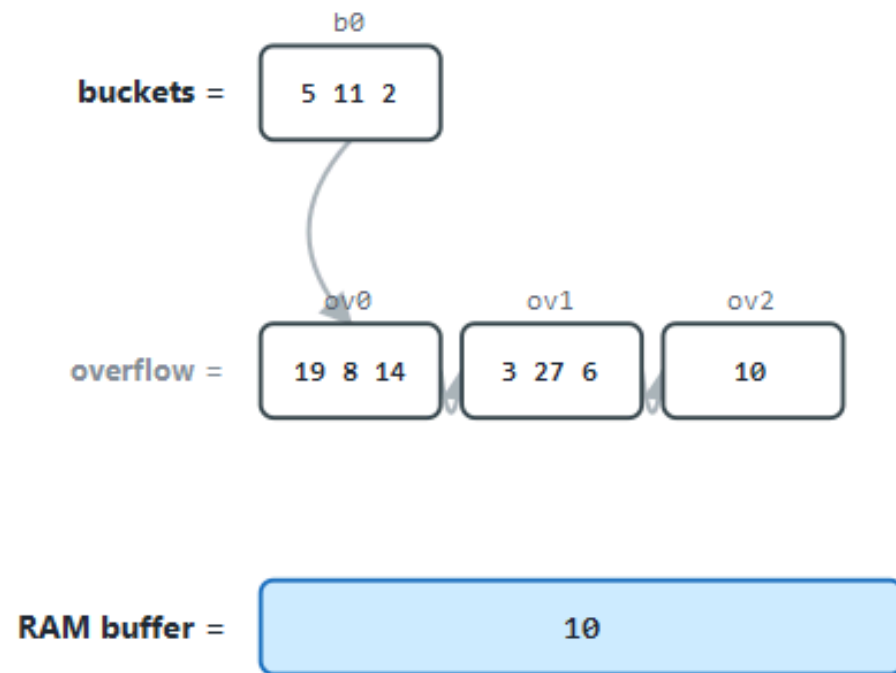
writes = 13
blocks used = 5
 $h(21)=21\%5=1$

overflow =



Done: 13 keys, 5 home bucket(s), 0 overflow block(s) (5 blocks in total), 13 writes. Load factor = $13 / (5 \times 3) = 0.87$.

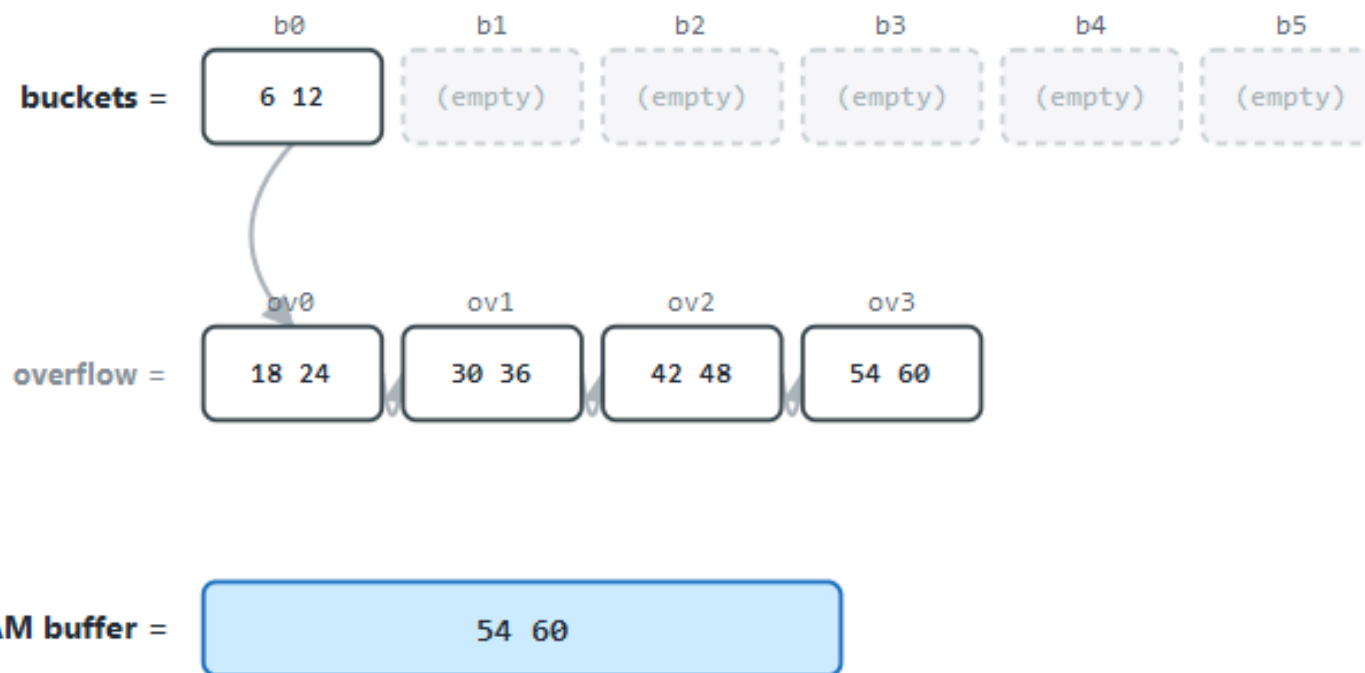
Edge case — $m=1$, everything chains



writes = 13
blocks used = 4
 $h(10)=10\%1=0$

Done: 10 keys, 1 home bucket(s), 3 overflow block(s) (4 blocks in total), 13 writes. Load factor = $10 / (1 \times 3) = 3.33$.

Edge case — all 10 keys hash to the same bucket



writes = 14
blocks used = 10
 $h(60) = 60 \% 6 = 0$

Done: 10 keys, 6 home bucket(s), 4 overflow block(s) (10 blocks in total), 14 writes. Load factor = $10 / (6 \times 2) = 0.83$.

Code — hash, then chain on overflow

```
int h = key % m, cur = h;
while (bucket[cur].count == bf) {
    if (bucket[cur].next < 0)
        bucket[cur].next = new_block();
    cur = bucket[cur].next;    /* follow chain */
}
bucket[cur].keys[bucket[cur].count++] = key;
```

Complexity

- Average case: **$O(1)$** block accesses, same as Week 6's chaining
- Plus one more access per overflow block that must be followed
- Stays small while the **load factor** $n / (m * bf)$ is reasonable
- Worst case ("m=1" above): degrades toward $O(n / bf)$

Common mistakes

- Chaining a new overflow block onto the **wrong** block
- Choosing `m` too small for the expected key count
- Forgetting a bucket's capacity is `bf`, not 1 — unlike Week 6

Mini-quiz

Is bucket chaining here closer to Week 6's separate chaining, or Week 6's open addressing? Why?

Answer

Separate chaining. A full bucket grows a chain of **additional blocks**, off to the side — never touching any other bucket, exactly like Week 6's per-slot linked list.

9. Progressive Overflow: Linear Probing on Disk

A question to start

Section 8's overflow blocks cost a whole extra block per overflow, however few keys it holds. Can probing avoid that extra block entirely?

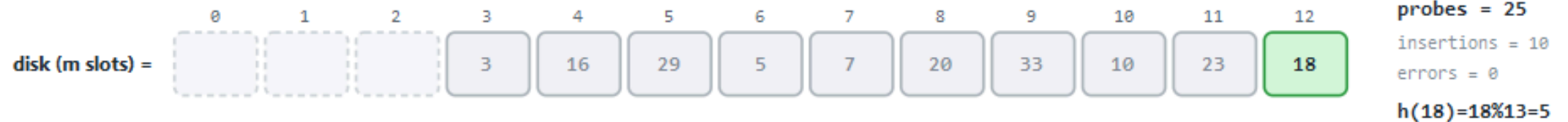
The idea: probe the next slot, wrap around

- Home slot: $h(\text{key}) = \text{key} \bmod m$
- Occupied by a different key? Try $(h+1) \bmod m$, then next
- $\bmod m$ makes the sequence **wrap around** past the last slot
- Stop: empty slot (insert), duplicate, or all m slots tried (full)

Intuition — parking in the nearest free spot

- Your assigned spot is taken? Take the **next** spot over
- Still taken? The next one — wrapping to spot 0 at the end
- No separate overflow lot at all — everyone parks in the same lot
- Exactly Week 6's open addressing, one level down, on disk

Progressive overflow, step by step



RAM: probing =

Done: 10 request(s), 10 inserted, 0 error(s) (duplicate/file full), 25 probes in total. As probe runs grow (primary clustering), later insertions get slower too.

Edge case — every key shares one home slot



RAM: probing =

Done: 10 request(s), 9 inserted, 1 error(s) (duplicate/file full), 54 probes in total. As probe runs grow (primary clustering), later insertions get slower too.

Edge case — the table is completely full



RAM: probing =

Done: 10 request(s), 9 inserted, 1 error(s) (duplicate/file full), 18 probes in total. As probe runs grow (primary clustering), later insertions get slower too.

Code — probe until room, duplicate, or full

```
int h = key % m, i = h, tries = 0;
while (tries < m) {
    if (table[i] == EMPTY) {
        table[i] = key; return i;
    }
    if (table[i] == key) return -2; /* dup */
    i = (i + 1) % m; tries++;
}
return -1; /* file genuinely full */
```

Complexity

- Close to $O(1)$ probes at a **low** load factor n / m
- Degrades sharply as load factor rises
- "Hard" scenario (91% loaded): up to **5** probes for one insert
- Worst case, nearly full: $O(m)$ probes

Common mistakes

- Not bounding the probe loop (`tries < m`): risks an infinite loop
- Treating "file full" as a crash instead of a normal outcome
- Forgetting `key % m` needs a guard for negative keys

Mini-quiz

What is "primary clustering", and why does the "same home slot" edge case demonstrate it at its most extreme?

Answer

A growing occupied run that keeps attracting more collisions. Here, every key shares one home slot, so probe counts grow 1, 2, 3, ... in lock-step — clustering at its most extreme.

10. Deletion with Tombstones

A question to start

If a deleted slot is simply cleared to `EMPTY`,
what happens to a later search for a *different*
key whose probe chain passed through it?

The idea: a tombstone means "keep looking"

- **Tombstone**: distinct from both `EMPTY` and any real key
- A search **skips over** a tombstone, exactly like any occupied slot
- Only a genuine `EMPTY` slot is allowed to **end** a search
- An **insert**, though, is free to treat a tombstone as reusable

Intuition — a "gone, but do not stop here" sign

- A parking spot's occupant left: put up a **temporary sign**
- The sign is not "empty" (keep circling) or "occupied" (keep out)
- It means: someone parked here once, your car might be further on
- A new car may still park here; the sign then comes down

Deletion with tombstones, step by step



RAM: probing =

Done: a tombstone marks a deleted slot as "not empty, something was here, keep searching" rather than truly empty — this keeps search correct, and insertion later reclaims the space by reusing it.

Edge case — a fully-tombstoned table



RAM: probing =

Done: a tombstone marks a deleted slot as "not empty, something was here, keep searching" rather than truly empty — this keeps search correct, and insertion later reclaims the space by reusing it.

Edge case — delete a key, then reinsert the same key



RAM: probing =

Done: a tombstone marks a deleted slot as "not empty, something was here, keep searching" rather than truly empty — this keeps search correct, and insertion later reclaims the space by reusing it.

Code — find must skip tombstones, not stop

```
int i = key % m, tries = 0;
while (tries < m) {
    if (table[i] == EMPTY) return -1;
    if (table[i] == key) return i;
    /* TOMB: fall through, keep probing */
    i = (i + 1) % m; tries++;
}
return -1;
```

Code — insert may reuse a tombstone

```
int i = key % m, tries = 0;
while (tries < m) {
    if (table[i] == EMPTY ||
        table[i] == TOMB) {
        table[i] = key; return i; /* reused! */
    }
    if (table[i] == key) return -2;
    i = (i + 1) % m; tries++;
}
```

Complexity

- `ts_find` / `ts_delete`: same **$O(\text{probe chain length})$** as section 9
- Tombstones add **no new asymptotic cost**
- `ts_insert`: often **less** in practice — may reuse a tombstone early
- Only *which* slots a search is willing to pass through changes

Common mistakes

- Deleting by simply clearing the slot to `EMPTY`
- Making `ts_find` stop at, or match, a tombstone
- Forgetting a fully-tombstoned table has **no** `EMPTY` to rely on

Mini-quiz

After `ts_insert` reuses a tombstone for a new key, why does a search for some other key that also probes past that slot still work?

Answer

Once reused, the slot holds a real key **again** — indistinguishable from any other occupied slot. Other keys' searches behave exactly as if the deletion never happened.

11–12. Comparing and Choosing

Comparison table

Organisation	Locate	Insert / Delete	Best for
Sequential (unsorted)	$O(\text{numBlocks})$	$O(1) / O(\text{numBlocks})$	Process every record anyway
Sequential (sorted)	$O(\log \text{ numBlocks})$	$O(\text{numBlocks})$ both	Batch updates (section 6)
Relative (direct)	$O(1)$	$O(1) / O(1)$, leaves a hole	RRN is naturally known
Hashing (buckets or probing)	$O(1)$ avg	$O(1)$ avg	Arbitrary keys, fast lookup

Choosing a technique (1/2)

Situation	Best choice
Process every record anyway	Sequential, any order
Batch of changes, periodically	Sequential (sorted) + transactions
Occasional lookups, rarely changes	Sequential (sorted), binary search
Records numbered 0, 1, 2, ...	Relative (direct) file

Choosing a technique (2/2)

Situation	Best choice
Arbitrary key, simplicity over compactness	Hashing, bucket chaining
Arbitrary key, compactness, rare deletes	Hashing, progressive overflow
Arbitrary key, frequent deletes	Progressive overflow + tombstones



Wrap-Up

Summary

- **Records/fields:** fixed, delimited, or length-prefixed
- **Blocking factor:** `bf` records share one disk access
- **Sequential:** search every block, or binary-search a sorted one
- **Sequential update:** merge master + transactions in one pass
- **Relative access:** `block = rrn/bf` — true $O(1)$
- **Hashing:** chain overflow blocks, or probe with tombstones

1. Why does this week count block reads, not comparisons?

A disk access costs vastly more than any RAM operation, and always transfers a WHOLE block — so block count, not comparison count, predicts real speed.

2. Why must a sorted file's binary search compare against a block's first/last key, not its middle key?

A whole BLOCK of records is being ruled in or out per comparison, and only the first/last key defines that block's range.

3. Why must a duplicate-add error still copy the original master record forward?

The master record did nothing wrong — only the transaction is rejected; not copying it forward would silently lose valid data.

4. Why can insert safely reuse a tombstone, when search must not stop at one?

Search must preserve every OTHER key's ability to be found; reuse by insert does not interfere, because the slot then just holds a new, real key.

5. Why does a relative file's $O(1)$ cost not depend on the file's total size at all?

One fseek, computed from rrn and bf alone, then one block read — nothing about that computation depends on how many OTHER records the file holds.

6. Why is chaining overflow blocks (section 8) a closer match to separate chaining than to open addressing?

A full bucket grows a chain of ADDITIONAL blocks off to the side, never touching any other bucket — exactly Week 6's separate chaining, one level down.

Next week

Week 14 — File Organisation II

Indexed sequential files · B-trees · extendible hashing · external sorting — extending this week's block-read cost model to bigger tools.

References (1/2)

- Course syllabus, Week 13: `docs/syllabus/syllabus.en.md`
- A. L. Tharp. *File Organization and Processing*.
John Wiley & Sons, 1988 — the primary textbook
- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*. MIT Press — hash table chapter

References (2/2)

- Knuth. *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed — file processing, hashing, and linear probing's historical analysis
- [williamfiset/Algorithms](#) · Programiz DSA